

Sincronización por variables compartidas

Cómo coordinar procesos que comparten memoria usando solo variables comunes: el problema de la sección crítica (locks) y la sincronización por barreras — con las definiciones, invariantes y algoritmos completos vistos en la teoría.

MEMORIA COMPARTIDA

[Variables compartidas](#)[Semáforos](#)[Monitores](#)

MEMORIA DISTRIBUIDA

[Mensajes asíncronos](#)[Mensajes sincrónicos](#)[RPC](#)[Rendezvous](#)[Sección crítica](#)[Sentencias await](#)[Locks – 6 soluciones](#)[Barreras – 4 soluciones](#)[Cierre](#)

PUNTO DE PARTIDA

El problema de la Sección Crítica

Implementar en software una acción atómica: garantizar que, mientras un proceso ejecuta su sección crítica (SC), ningún otro proceso pueda hacerlo. Toda solución a este problema sirve además para implementar sentencias await arbitrarias — por eso la SC es la base de todo lo que sigue en la materia.

```
process SC[i = 1 to n] {  
  while (true) {  
    protocolo de entrada;    // <  
    sección crítica;          // SC  
    protocolo de salida;     // >  
    sección no crítica;
```

```
}  
}
```

Hay una **solución trivial**: envolver la SC entera en una acción atómica, $\langle S \rangle \Rightarrow$ `SCEnter; S; SCExit`; . El problema real es cómo implementar `SCEnter` y `SCExit` — es decir, los protocolos de entrada y salida — de forma que valgan las cuatro propiedades siguientes.

01

Exclusión mutua

A lo sumo un proceso está en su sección crítica en cualquier instante.

02

Ausencia de deadlock (livelock)

Si dos o más procesos intentan entrar a su SC, al menos uno lo logra.

03

Ausencia de demora innecesaria

Si un proceso quiere entrar y los demás están en su SNC o terminaron, nada se lo impide.

04

Eventual entrada

Todo proceso que intenta entrar a su SC tiene posibilidades de lograrlo — eventualmente lo hará. Depende de la política de scheduling.

Busy waiting: técnica base de todas las soluciones que siguen — un proceso chequea repetidamente una condición hasta que se vuelve verdadera. Ventaja: se puede implementar con instrucciones de cualquier procesador. Desventaja: ineficiente en multiprogramación, porque el procesador se desperdicia "girando" en lugar de ejecutar otro proceso útil. Es aceptable si cada proceso corre en su propio procesador.

Corolario

Implementación de sentencias `await`

Cualquier solución al problema de la SC sirve para implementar acciones atómicas más generales, incondicionales o condicionales, reemplazando el protocolo de entrada/salida por `SCEnter` / `SCExit`.

Acción atómica incondicional

$\langle S \rangle$

```
SCEnter; S; SCExit;
```

Acción atómica condicional

`await (B) S;` — correcto pero ineficiente

```
SCEnter;
while (not B) { SCExit; SCEnter; }
S; SCExit;
```

Para reducir contención de memoria

se agrega un Delay entre reintentos

```
SCEnter;
while (not B) { SCExit; Delay; SCEnter; }
S; SCExit;
```

Caso particular

S es skip y B se evalúa atómicamente

```
await (B); ≡ while (not B) skip;
```

LOCKS

Seis soluciones, cada vez más finas

De una solución que solo sirve en monoprocesador a algoritmos que garantizan fairness sin instrucciones especiales de hardware. Cada una corrige una debilidad de la anterior.

1

Deshabilitar interrupciones

solución HW

El protocolo de entrada apaga las interrupciones del procesador; el de salida las reactiva. Mientras están deshabilitadas, ningún otro proceso puede tomar el control del procesador.

```
process SC[i=1 to n] {
  while (true) {
    deshabilitar interrupciones; // protocolo de entrada
    sección crítica;
    habilitar interrupciones;    // protocolo de salida
    sección no crítica;
  }
}
```

Correcta en un monoprocesador, pero penaliza la performance: durante toda la SC no se usa la multiprogramación.

× No es correcta en un multiprocesador — deshabilitar interrupciones en un procesador no impide a los demás entrar a la SC.

2

Grano grueso

await no atómico

Primer intento, sin protección — cada proceso simplemente marca su bandera al entrar. **No asegura el invariante MUTEX**, porque nada impide que ambas banderas estén en true a la vez:

VERSIÓN INGENUA – MUTEX: $\neg(IN1 \wedge IN2)$

```
bool in1=false, in2=false;
process SC1 { while(true){ in1=true; SC; in1=false; SNC; } }
process SC2 { while(true){ in2=true; SC; in2=false; SNC; } }
```

VERSIÓN CORREGIDA – CON AWAIT

```
process SC1 { while(true){
  await (not in2) in1=true;
  sección crítica;
  in1=false;
  sección no crítica;
} }
process SC2 { while(true){
  await (not in1) in2=true;
  sección crítica;
  in2=false;
  sección no crítica;
} }
```

- ✓ **Exclusión mutua:** por construcción, SC1 y SC2 se excluyen en el acceso a la SC.
- ✓ **Ausencia de deadlock:** si lo hubiera, ambos estarían bloqueados en su entrada $\Rightarrow$ in1 e in2 serían true a la vez, y eso no puede darse (son false al inicio y al salir de la SC).
- ✓ **Ausencia de demora innecesaria:** si SC1 está fuera de su SC, in1 es false — y SC2 nunca queda esperando sin motivo.
- ✓ **Eventual entrada:** se cumple solo con un scheduling *fuertemente fair* (todo proceso en SC eventualmente sale, liberando la guarda del otro).

GENERALIZACIÓN – DE DOS BANDERAS A UN LOCK, Y LUEGO A N PROCESOS

```
bool lock=false;                                // lock = in1 v in2
process SC[i=1..n] { while(true){
    await (not lock) lock=true;
    sección crítica;
    lock=false;
    sección no crítica;
} }
```

3

Spin locks

grano fino

El objetivo es hacer atómico el await de grano grueso. Se logra con una instrucción de hardware que lee y modifica en una única acción atómica: Test&Set, Fetch&Add o Compare&Swap.

TEST & SET

```
bool TS (bool ok) {
    < bool inicial = ok;
    ok = true;
    return inicial; >
}
```

SOLUCIÓN CON SPIN LOCK

```
bool lock=false;
process SC[i=1 to n] { while(true){
    while (TS(lock)) skip;    // spinning
    sección crítica;
    lock=false;
    sección no crítica;
} }
```

Los procesos quedan "girando" (spinning) mientras esperan que lock se limpie. Cumple las 4 propiedades si el scheduling es fuertemente fair; una política débilmente fair alcanza en la práctica, porque rara vez todos los procesos intentan entrar a la vez.

MEJORA – TEST-AND-TEST-AND-SET

Test&Set siempre *escribe* en lock, aunque el valor no cambie, lo que genera tráfico de memoria innecesario. Conviene primero leer sin modificar, y solo intentar el TS cuando parece libre:

```
while (lock) skip;
while (TS(lock))
    while (lock) skip;
```

⚠ La contención de memoria se reduce, pero no desaparece: cuando lock pasa a false, posiblemente todos intenten TS al mismo tiempo.

Algoritmo Tie-Breaker

fair, sin HW especial

Los spin locks no controlan el orden de los procesos demorados: con un scheduling que no sea fuertemente fair, alguno podría no entrar nunca. Tie-Breaker exige solo scheduling *débilmente* fair y no usa instrucciones especiales — a cambio, es más complejo.

Cada proceso tiene una variable que indica que comenzó su protocolo de entrada, más una variable compartida ultimo que rompe empates: indica quién fue el último en comenzar, y a ese se le quita prioridad (se lo demora).

GRANO GRUESO — 2 PROCESOS

```
bool in1=false, in2=false;
int ultimo=1;
process SC1 { while(true){
    ultimo=1; in1=true;
    await (not in2 or ultimo==2);
    sección crítica;
    in1=false; sección no crítica;
} }
process SC2 { while(true){
    ultimo=2; in2=true;
    await (not in1 or ultimo==1);
    sección crítica;
    in2=false; sección no crítica;
} }
```

GRANO FINO — 2 PROCESOS (BUSY WAITING)

```
process SC1 { while(true){
    in1=true; ultimo=1;
    while (in2 and ultimo==1) skip;
    sección crítica;
    in1=false; sección no crítica;
} }
process SC2 { while(true){
    in2=true; ultimo=2;
    while (in1 and ultimo==2) skip;
    sección crítica;
```

```
in2=false; sección no crítica;
} }
```

GENERALIZACIÓN A N PROCESOS

```
int in[1:n] = ([n] 0), ultimo[1:n] = ([n] 0);
process SC[i = 1 to n] { while (true) {
  for [j = 1 to n] { // n-1 etapas
    in[i] = j; ultimo[j] = i;
    for [k = 1 to n st i <> k] {
      while (in[k] >= in[i] and ultimo[j]==i) skip;
    }
  }
  sección crítica;
  in[i] = 0;
  sección no crítica;
} }
```

Con n procesos, el protocolo de entrada recorre $n-1$ etapas; en cada una, una instancia de tie-breaker de a pares decide quiénes avanzan. Si a lo sumo un proceso a la vez puede completar las $n-1$ etapas, a lo sumo uno está en la SC.

5

Algoritmo Ticket

requiere Fetch&Add

Tie-Breaker para n procesos es complejo y costoso en tiempo. Ticket resuelve esto como en una carnicería: cada proceso saca un número mayor que el de cualquier otro que ya espera, y luego espera a que le toque su turno.

```
int numero=1, proximo=1, turno[1:n] = ([n] 0);
{ TICKET: proximo>0 ∧
  (∀i: (SC[i] está en su SC) ⇔ (turno[i]==proximo ∧ turno[i]>0))
  ∧ (∀i≠j: turno[i] ≠ turno[j]) }

process SC[i:1..n] { while(true){
  < turno[i]=numero; numero=numero+1; >
  < await turno[i]==proximo; >
  sección crítica;
  < proximo=proximo+1; >
  sección no crítica;
} }
```

- ✓ **Exclusión mutua:** TICKET es un invariante global — numero se lee e incrementa atómicamente, proximo se incrementa atómicamente ⇒ a lo sumo un proceso en SC.

- ✓ **Ausencia de deadlock y de demora innecesaria:** resultan de que los valores de turno son todos únicos.
- ✓ **Eventual entrada:** asegurada con scheduling débilmente fair.

⚠ número y próximo crecen sin límite. En la práctica podrían resetearse a un valor chico cuando sea seguro hacerlo.

¿CÓMO SE ASIGNA EL NÚMERO ATÓMICAMENTE? – FETCH & ADD

```
FA (var, incr): < temp = var; var = var + incr; return (temp); >

int numero=1, proximo=1, turno[1:n] = ([n] 0);
process SC[i:1..n] { while(true){
  turno[i] = FA(numero, 1);
  while (turno[i] <> proximo) skip;
  sección crítica;
  proximo = proximo + 1;
  sección no crítica;
} }
```

El await puede implementarse con busy waiting porque la condición referencia una sola variable compartida; el incremento de proximo puede ser un load/store normal, porque a lo sumo un proceso ejecuta su protocolo de salida a la vez.

6

Algoritmo Bakery

fair, sin HW ni contador global

Si no existe una instrucción Fetch&Add, Ticket debería simularse con una SC propia, y la solución podría dejar de ser fair. Bakery evita depender de un contador global: cada proceso recorre los números de *todos los demás* y se autoasigna uno mayor; después espera a tener el número más chico entre los que esperan. Los procesos se comparan entre sí, no contra un valor central.

```
int turno[1:n] = ([n] 0);
{ BAKERY:  $\forall i: (SC[i] \text{ está en su SC}) \implies (\text{turno}[i] > 0) \wedge$ 
   $(\forall j \neq i: \text{turno}[j] = 0 \vee \text{turno}[i] < \text{turno}[j])$  }
```

VERSIÓN NO IMPLEMENTABLE DIRECTAMENTE

```
process SC[i=1 to n] { while(true){
  turno[i] = max(turno[1:n]) + 1;
  for [j=1 to n st j <> i]
    await (turno[j]==0 or turno[i]<turno[j]);
  sección crítica;
  turno[i] = 0;
}
```

```
sección no crítica;
} }
```

⚠ Dos problemas: calcular `turno[i]` exige el máximo de n valores (no es una acción atómica simple), y el `await` referencia una variable compartida dos veces.

VERSIÓN DE GRANO FINO (IMPLEMENTABLE)

```
process SC[i=1 to n] { while(true){
  turno[i] = 1; // indica que comenzó a entrar
  turno[i] = max(turno[1:n]) + 1;
  for [j=1 to n st j != i] // espera su turno
    while (turno[j]!=0) and ((turno[i],i) > (turno[j],j)) skip;
  sección crítica;
  turno[i] = 0;
  sección no crítica;
} }
```

Comparar el par `(turno[i], i)` en vez de solo `turno[i]` desempata cuando dos procesos sacan el mismo número: gana el de menor índice. Es fair y no requiere instrucciones especiales ni contador global — pero es, de las seis, la solución más compleja de implementar correctamente.

BARRERAS

Sincronización Barrier

Una barrera es un punto de demora al que deben llegar *todos* los procesos antes de que cualquiera pueda continuar. En algoritmos iterativos la barrera se reutiliza en cada vuelta, lo que complica algunas de las soluciones más simples.

1

Contador compartido

requiere FA

Cada proceso incrementa una variable compartida cantidad al llegar a la barrera; cuando cantidad llega a n , todos pueden pasar.

```
int cantidad=0;
process Worker[i=1 to n] { while(true){
  código para implementar la tarea i;
  cantidad = cantidad + 1;
  await (cantidad == n);
} }
```

VERSIÓN CON FETCH & ADD

```
int cantidad=0;
process Worker[i=1 to n] { while(true){
  código para implementar la tarea i;
  FA(cantidad, 1);
  while (cantidad <> n) skip;
} }
```

⚠ Pregunta abierta en la teoría: ¿cuándo se reinicia cantidad a 0 para la siguiente iteración? Resolverlo bien es lo que motiva las siguientes soluciones.

2

Flags y Coordinador

proceso extra

Si no existe Fetch&Add, cantidad puede distribuirse en un arreglo `arribo[1..n]`, pero sumarlo todo en el `await (arribo[1]+...+arribo[n]==n)` reintroduce contención de memoria y es ineficiente. La alternativa: un proceso Coordinador aparte, que espera cada llegada y recién entonces habilita a todos a seguir — cada Worker espera por un único valor.

```
int arribo[1:n]=([n] 0), continuar[1:n]=([n] 0);

process Worker[i=1 to n] { while(true){
  código para implementar la tarea i;
  arribo[i] = 1;
  await (continuar[i] == 1);
  continuar[i] = 0;
} }

process Coordinador { while(true){
  for [i=1 to n] { await (arribo[i]==1); arribo[i]=0; }
  for [i=1 to n] continuar[i] = 1;
} }
```

⚠ Requiere un proceso (y procesador) extra, y el tiempo de ejecución del Coordinador es proporcional a n .

3

Árboles

combining tree

Para eliminar el proceso extra y el costo $O(n)$ del Coordinador, se combinan los roles: cada Worker actúa también de coordinador de una parte de los demás. Los Workers se organizan en forma de árbol — las señales de arribo suben desde las hojas hacia la raíz, y las señales de "continuar" bajan desde la raíz hacia las hojas.

Es la *combining tree barrier*: más eficiente que un coordinador único cuando n es grande, porque el trabajo de combinar llegadas se reparte entre varios procesos en paralelo en vez de recaer en uno solo.

Barrera simétrica — Butterfly barrier

$O(\log n)$ etapas

A diferencia del árbol combinador, aquí todos los procesos juegan el *mismo* rol. Se parte de una barrera simple entre dos procesos:

```
W[i]:: await (arribo[i]==0);    W[j]:: await (arribo[j]==0);
      arribo[i]=1;              arribo[j]=1;
      await (arribo[j]==1);      await (arribo[i]==1);
      arribo[j]=0;              arribo[i]=0;
```

Si n es potencia de 2, estas barreras de a pares se combinan en $\log_2 n$ etapas: en la etapa s , cada worker sincroniza con otro ubicado a distancia 2^{s-1} . Cuando cada worker completó sus $\log_2 n$ etapas, todos pueden continuar.

```
int E = log(N);
int arribo[1:N] = ([N] 0);
process P[i=1..N] { int j;
  while (true) {
    // código anterior a la barrera
    for (etapa=1; etapa<=E; etapa++) {
      j = (i-1) XOR (1<<(etapa-1)); // con quién sincronizo
      while (arribo[i]==1) skip;
      arribo[i]=1;
      while (arribo[j]==0) skip;
      arribo[j]=0;
    }
    // código posterior a la barrera
  }
}
```

CIERRE

Defectos de la sincronización por busy waiting

- **Protocolos complejos:** no separan con claridad las variables de sincronización de las que se usan para computar resultados.
- **Difíciles de** probar su corrección se complica a medida que crece el número de procesos — incluso la verificación formal se vuelve costosa.

verificar:

- **Ineficientes en multiprogramación:** un procesador haciendo spinning podría estar ejecutando productivamente otro proceso.

→

Necesidad de herramientas de sincronización de más alto nivel — próxima clase: **Semáforos** (Teoría 3).

Basado en teoria2.pdf – Programación Concurrente, Facultad de Informática UNLP.